МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Отчет по лабораторной работе ЛР1 по курсу

«Парадигмы и конструкции языков программирования»

Тема: «Основные конструкции языка Python»

Выполнил:
Студент группы ИБМ3-34Б

Норжилова Н. З.
Преподаватель:
Бурмистрова М.Ю

Правдина А.Д.

Задание:

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.

2. Программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).

3. Коэффициенты A, B, C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.

4. Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.

6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования.

Текст программы на Python:

```python
import sys
import math

def read_coefficient(name):
    while True:
        stroka = input(f"Введите {name}: ")
        try:
            value = float(stroka)
        except ValueError:
            print("Некорректное значение, повторите")
            continue
        return value


def read_params():
    a = None
    if len(sys.argv) > 1:
        try:
            value = float(sys.argv[1])
            if value != 0:
                a = value
        except ValueError:
            pass
    if a is None:
        a = read_coefficient("A")
```

```
    b = None
    if len(sys.argv) > 2:
        try:
            b = float(sys.argv[2])
        except ValueError:
            pass
    if b is None:
        b = read_coefficient("B")

    c = None
    if len(sys.argv) > 3:
        try:
            c = float(sys.argv[3])
        except ValueError:
            pass
    if c is None:
        c = read_coefficient("C")

    return a, b, c

def list_roots_square(b,c):
    if b == 0:
        if c == 0:
            print("Уравнение имеет бесконечно много решений (0 = 0).")
            return []
        else:
            print("Уравнение не имеет решений.")
            return []

    right_square = -c / b
    if right_square < 0:
        return []
    if right_square == 0:
        return [0.0]
    r = math.sqrt(right_square)
    return [r, -r]

def list_roots(a, b, c):
    if a==0:
        return list_roots_square(b,c)
    d = b * b - 4 * a * c
    roots = []
    if d < 0:
        return []
    if d == 0:
        t = -b / (2 * a)
        if t < 0:
            return []
        if t == 0:
            roots = [0.0]
```

```
        else:
            roots = [math.sqrt(t), -math.sqrt(t)]
    else:
        t1 = (-b - math.sqrt(d)) / (2 * a)
        t2 = (-b + math.sqrt(d)) / (2 * a)
        for t in (t1, t2):
            if t < 0:
                continue
            if t == 0:
                roots.append(0.0)
            else:
                roots.append(math.sqrt(t))
                roots.append(-math.sqrt(t))

    return list(set(roots))

def print_roots(roots):
    if not roots:
        print("нет корней")
    else:
        print(roots)

def main():
    a, b, c = read_params()
    roots = list_roots(a, b, c)
    print_roots(roots)

main()
```

Пример выполнения программы:

Введите A: 5

Введите B: -10

Введите C: 5

[-1, 1]

Текст программы на C++:

```
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
#include <set>

class Coefficient {
private:
    std::string name;
```

```
    int index;
    double value;
    bool readFromArgv(int argc, char* argv[], double& out) const {
        if (argc <= index) {
            return false;
        }
        try {
            std::string s(argv[index]);
            size_t pos;
            double v = std::stod(s, &pos);
            if (pos != s.size()) {
                return false;
            }
            out = v;
            return true;
        } catch (const std::exception&) {
            return false;
        }
    }

    double readFromConsole() const {
        while (true) {
            std::cout << "Введите " << name << ": ";
            std::string line;
            std::getline(std::cin, line);
            try {
                size_t pos;
                double v = std::stod(line, &pos);
                if (pos != line.size()) {
                    throw std::invalid_argument("trailing");
                }
                return v;
            } catch (const std::exception&) {
                std::cout << "Некорректное значение, повторите\n";
            }
        }
    }

public:
    Coefficient(const std::string& n, int idx)
        : name(n), index(idx), value(0.0) {}

    void read(int argc, char* argv[]) {
        double v;
        if (readFromArgv(argc, argv, v)) {
            value = v;
        } else {
            value = readFromConsole();
        }
    }
```

```
    double getValue() const {
        return value;
    }
};


class BiquadraticEquation {
private:
    double a, b, c;
    std::vector<double> solveSquare() const {
        std::vector<double> roots;

        if (b == 0) {
            if (c == 0) {
                std::cout << "Уравнение имеет бесконечно много решений (0 =
0).\n";
            } else {
                std::cout << "Уравнение не имеет решений.\n";
            }
            return roots;
        }

        double x_squared = -c / b;
        if (x_squared < 0) {
            return roots;
        }
        if (x_squared == 0) {
            roots.push_back(0.0);
            return roots;
        }
        double r = std::sqrt(x_squared);
        roots.push_back(r);
        roots.push_back(-r);
        return roots;
    }

public:
    BiquadraticEquation() : a(0), b(0), c(0) {}

    void readCoefficients(int argc, char* argv[]) {
        Coefficient coeffA("A", 1);
        coeffA.read(argc, argv);
        a = coeffA.getValue();

        Coefficient coeffB("B", 2);
        coeffB.read(argc, argv);
        b = coeffB.getValue();

        Coefficient coeffC("C", 3);
        coeffC.read(argc, argv);
        c = coeffC.getValue();
```

```
    }

    std::vector<double> solve() const {
        if (a == 0) {
            return solveSquare();
        }
        double d = b * b - 4 * a * c;
        std::vector<double> roots;

        if (d < 0) {
            return {};
        }

        if (d == 0) {
            double t = -b / (2 * a);
            if (t < 0) {
                return {};
            }
            if (t == 0) {
                roots.push_back(0.0);
            } else {
                roots.push_back(std::sqrt(t));
                roots.push_back(-std::sqrt(t));
            }
        } else {
            double t1 = (-b - std::sqrt(d)) / (2 * a);
            double t2 = (-b + std::sqrt(d)) / (2 * a);
            for (double t : {t1, t2}) {
                if (t < 0) continue;
                if (t == 0) {
                    roots.push_back(0.0);
                } else {
                    roots.push_back(std::sqrt(t));
                    roots.push_back(-std::sqrt(t));
                }
            }
        }

        std::set<double> uniqueRoots(roots.begin(), roots.end());
        return std::vector<double>(uniqueRoots.begin(), uniqueRoots.end());
    }

    void printRoots() const {
        std::vector<double> roots = solve();
        if (roots.empty()) {
            std::cout << "нет корней\n";
            return;
        }
        std::cout << "[";
        for (size_t i = 0; i < roots.size(); ++i) {
            if (i > 0) std::cout << ", ";
```

```
            std::cout << roots[i];
        }
        std::cout << "]\n";
    }
};


int main(int argc, char* argv[]) {
    BiquadraticEquation eq;
    eq.readCoefficients(argc, argv);
    eq.printRoots();
    return 0;
}
```

Пример выполнения программы:

Введите A: 5

Введите B: -10

Введите C: 5

[-1, 1]